

DSA STUDY BLUEPRINT SERIES

30. Spiral Matrix Leetcode 54

Printing 2D Matrix boundaries in Spiral loops

Section 04 • C++ Core Data Structures & Algorithms

Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Bounds Tracking:** Maintain 4 boundaries: `top`, `bottom`, `left`, and `right`.
- Shift bounds inwards after completing each outer loop boundary traversal.

Memory & Execution Blueprint

Spiral Loops direction path:

[Right] → [Down] → [Left] → [Up] ⇒ Shrink Bounds

C++ Implementation Reference

Standard code layout and syntax

```
vector spiralOrder(vector<vector<int>>& matrix) {  
    int top = 0, bottom = matrix.size() - 1;  
    int left = 0, right = matrix[0].size() - 1;  
    vector ans;  
    while (top <= bottom && left <= right) {  
        for (int i = left; i <= right; i++) ans.push_back(matrix[top][i]); top++;  
        for (int i = top; i <= bottom; i++) ans.push_back(matrix[i][right]); right--;  
        if (top <= bottom) {  
            for (int i = right; i >= left; i--) ans.push_back(matrix[bottom][i]); bottom--;  
        }  
        if (left <= right) {  
            for (int i = bottom; i >= top; i--) ans.push_back(matrix[i][left]); left++;  
        }  
    }  
    return ans;  
}
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Time Complexity	$O(R * C)$
Space Complexity	$O(1)$ (excluding result)

Key Practices & Gotchas

- **Important:** Guard boundary indices inside loops to prevent double printing in single row/column grids.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's dry run Row-Major index resolution mapping for element at 2D coordinate `[1][2]` (3x3 grid):

Row Index (i)	Col Index (j)	Formula: <code>index = i * cols + j</code>	Resolved 1D Offset
1	2	<code>index = 1 * 3 + 2</code>	Offset = 5 in linear memory block

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Row-Major Address Index mapping:** Flat memory locality makes sequential row scan speeds far faster than column-major scans.
- **Spiral Boundaries:** Keep strict checks inside individual boundary loop passes to avoid double print coordinates.
- **Related LeetCode Problems:** #54 Spiral Matrix, #74 Search a 2D Matrix, #48 Rotate Image.